4.6 SDK

Traducción de la sección 4.6 del manual GIM6010-8 rev 2.2 (págs. 85–94 del original). El código de ejemplo se reproduce tal cual aparece en el manual.

4.6.1 SDK de C/C++

Basado en el protocolo CAN Simple, puede descargarse el siguiente SDK de C/C++ para la integración: `https://gitee.com/cyberbeast/mwmotorsdk`. Siga los ejemplos de prueba detallados de ese repositorio para hacer las pruebas de integración.

4.6.2 SDK de Python

Instale primero `odrivetool` siguiendo los pasos del apartado 3.1 (`pip install --upgrade odrive`). Consulte el apartado 3.1.8: todos los comandos que allí se describen pueden usarse para desarrollar en Python.

A continuación se dan tres ejemplos.

1) Caso práctico: calibración tras dar tensión

```
import odrive
import time

odrv0 = odrive.find_any()
odrive.utils.dump_errors(odrv0)
odrv0.clear_errors()
odrv0.axis0.requested_state = odrive.utils.AxisState.MOTOR_CALIBRATION
time.sleep(5)
while (odrv0.axis0.current_state != 1):
    time.sleep(0.5)
odrive.utils.dump_errors(odrv0)
odrv0.axis0.requested_state = odrive.utils.AxisState.ENCODER_OFFSET_CALIBRATION
time.sleep(6)
while (odrv0.axis0.current_state != 1):
    time.sleep(0.5)
odrive.utils.dump_errors(odrv0)
odrv0.axis0.motor.config.pre_calibrated = 1
odrv0.axis0.encoder.config.pre_calibrated = 1
odrv0.save_configuration()
```

2) Caso práctico: control de velocidad

```
import odrive
import time

odrv0 = odrive.find_any()
odrv0.axis0.controller.config.control_mode = odrive.utils.ControlMode.VELOCITY_CONTROL
odrv0.axis0.controller.config.input_mode = odrive.utils.InputMode.VEL_RAMP
odrv0.axis0.controller.config.vel_ramp_rate = 50
odrv0.axis0.requested_state = odrive.utils.AxisState.CLOSED_LOOP_CONTROL
odrv0.axis0.controller.input_vel = 15
odrive.utils.dump_errors(odrv0)
```

```
time.sleep(5)
odrv0.axis0.controller.input_vel = 0
```

3) Caso práctico: control de posición

```
import odrive

odrv0 = odrive.find_any()
odrv0.axis0.controller.config.control_mode = odrive.utils.ControlMode.POSITION_CONTROL
odrv0.axis0.controller.config.input_mode = odrive.utils.InputMode.POS_FILTER
odrv0.axis0.requested_state = odrive.utils.AxisState.CLOSED_LOOP_CONTROL
odrv0.axis0.controller.input_pos = 10
```

4) Caso práctico: captura de datos

Durante el desarrollo y la integración es habitual necesitar capturar datos del funcionamiento del motor: la evolución de la tensión y la corriente, de la posición y la velocidad, etc. El SDK de Python integra una potente capacidad de captura de datos que permite recoger grandes cantidades de datos de funcionamiento con un script sencillo, lo que facilita mucho el desarrollo y la integración.

El código siguiente parte del ejemplo anterior de control de posición y le añade la captura en tiempo real de la posición y la corriente, guardando los datos en un archivo CSV.

```
import odrive
import numpy as np

odrv0 = odrive.find_any()
cap = odrive.utils.BulkCapture(
    lambda: [odrv0.axis0.motor.current_control.Iq_measured,
             odrv0.axis0.encoder.pos_estimate],
    data_rate=500, duration=2.5)
odrv0.axis0.controller.config.control_mode = odrive.utils.ControlMode.POSITION_CONTROL
odrv0.axis0.controller.config.input_mode = odrive.utils.InputMode.POS_FILTER
odrv0.axis0.requested_state = odrive.utils.AxisState.CLOSED_LOOP_CONTROL
odrv0.axis0.controller.input_pos = 10
np.savetxt("d:/test.csv", cap.data, delimiter=',')
```

En la llamada a `BulkCapture`, `data_rate` es la frecuencia de muestreo en Hz y `duration` el tiempo de muestreo en segundos; dentro de la expresión `lambda` puede introducirse cualquier operación matemática, lo que facilita el análisis de los datos.

4.6.3 SDK de Arduino

Con Arduino se puede controlar el motor a través del bus CAN; el protocolo subyacente es el descrito en el apartado 4.1. Hardware y bibliotecas compatibles:

- Placas Arduino con interfaz CAN integrada, como la Arduino UNO R4 Minima o la Arduino UNO R4 WiFi.
- Placas Teensy con interfaz CAN integrada (Teensy 4.0 y Teensy 4.1), a las que se accede con la biblioteca FlexCAN_T4 correspondiente.
- Otras placas compatibles con Arduino, mediante una placa de expansión CAN basada en el MCP2515.

El siguiente ejemplo muestra cómo configurar el motor para que responda a los comandos de posición del Arduino.

➢ Configurar el motor

Además de la configuración básica del apartado 3.1.3, fije el ancho de banda de control en 20 rad/s: la velocidad de envío del Arduino Uno es limitada, así que no conviene un ancho de banda alto; con un Arduino más rápido puede aumentarse este valor.

➢ Configurar el CAN

Configure el CAN según se indica.

> **Nota del traductor:** el manual original no incluye aquí los pasos concretos de configuración del motor ni del CAN; solo los enuncia.

➢ Instalar la biblioteca ODriveArduino

Instale la biblioteca ODriveArduino siguiendo estos pasos (se supone que el usuario ya tiene instalado el IDE de Arduino):

1. Abra el IDE de Arduino.
2. Vaya a Sketch → Include Library → Manage Libraries.
3. Escriba «ODriveArduino» para buscarla.
4. Pulse sobre la biblioteca ODriveArduino encontrada para instalarla.

➢ Código fuente de Arduino

```
#include <Arduino.h>
#include "ODriveCAN.h"

// Documentation for this example can be found here:
// https://docs.odriverobotics.com/v/latest/guides/arduino-can-guide.html

/* Configuration of example sketch -------------------------------------------*/

// CAN bus baudrate. Make sure this matches for every device on the bus
#define CAN_BAUDRATE 500000

// ODrive node_id for odrv0
#define ODRV0_NODE_ID 0

// Uncomment below the line that corresponds to your hardware.
// See also "Board-specific settings" to adapt the details for your hardware
setup.

// #define IS_TEENSY_BUILTIN // Teensy boards with built-in CAN interface (e.g.
Teensy 4.1). See below to select which interface to use.
// #define IS_ARDUINO_BUILTIN // Arduino boards with built-in CAN interface (e.g.
Arduino Uno R4 Minima)
// #define IS_MCP2515 // Any board with external MCP2515 based extension module.
See below to configure the module.

/* Board-specific includes ---------------------------------------------------*/

#if defined(IS_TEENSY_BUILTIN) + defined(IS_ARDUINO_BUILTIN) +
defined(IS_MCP2515) != 1
#warning "Select exactly one hardware option at the top of this file."
```

```
#if CAN_HOWMANY > 0 || CANFD_HOWMANY > 0
#define IS_ARDUINO_BUILTIN
#warning "guessing that this uses HardwareCAN"
#else
#error "cannot guess hardware version"

#endif

#endif

#ifdef IS_ARDUINO_BUILTIN
// See https://github.com/arduino/ArduinoCore-API/blob/master/api/HardwareCAN.h
// and https://github.com/arduino/ArduinoCore-
renesas/tree/main/libraries/Arduino_CAN

#include <Arduino_CAN.h>
#include <ODriveHardwareCAN.hpp>
#endif // IS_ARDUINO_BUILTIN

#ifdef IS_MCP2515
// See https://github.com/sandeepmistry/arduino-CAN/
#include "MCP2515.h"
#include "ODriveMCPCAN.hpp"
#endif // IS_MCP2515

#ifdef IS_TEENSY_BUILTIN
// See https://github.com/tonton81/FlexCAN_T4
// clone https://github.com/tonton81/FlexCAN_T4.git into /src
#include <FlexCAN_T4.h>
#include "ODriveFlexCAN.hpp"
struct ODriveStatus; // hack to prevent teensy compile error
#endif // IS_TEENSY_BUILTIN

/* Board-specific settings ------------------------------------------------*/

/* Teensy */

#ifdef IS_TEENSY_BUILTIN

FlexCAN_T4<CAN1, RX_SIZE_256, TX_SIZE_16> can_intf;

bool setupCan() {
 can_intf.begin();
 can_intf.setBaudRate(CAN_BAUDRATE);
 can_intf.setMaxMB(16);
 can_intf.enableFIFO();

    can_intf.enableFIFOInterrupt();
    can_intf.onReceive(onCanMessage);
    return true;
}

#endif // IS_TEENSY_BUILTIN

/* MCP2515-based extension modules -*/

#ifdef IS_MCP2515

MCP2515Class& can_intf = CAN;
```

```
// chip select pin used for the MCP2515
#define MCP2515_CS 10

// interrupt pin used for the MCP2515
// NOTE: not all Arduino pins are interruptable, check the documentation for your
board!
#define MCP2515_INT 2

// freqeuncy of the crystal oscillator on the MCP2515 breakout board.
// common values are: 16 MHz, 12 MHz, 8 MHz
#define MCP2515_CLK_HZ 8000000

static inline void receiveCallback(int packet_size) {
    if (packet_size > 8) {
        return; // not supported
    }
    CanMsg msg = {.id = (unsigned int)CAN.packetId(), .len = (uint8_t)packet_size};
    CAN.readBytes(msg.buffer, packet_size);
    onCanMessage(msg);
}

bool setupCan() {
    // configure and initialize the CAN bus interface
    CAN.setPins(MCP2515_CS, MCP2515_INT);
    CAN.setClockFrequency(MCP2515_CLK_HZ);
    if (!CAN.begin(CAN_BAUDRATE)) {
        return false;
    }

    CAN.onReceive(receiveCallback);
    return true;
}

#endif // IS_MCP2515

/* Arduinos with built-in CAN */

#ifdef IS_ARDUINO_BUILTIN

HardwareCAN& can_intf = CAN;

bool setupCan() {
    return can_intf.begin((CanBitRate)CAN_BAUDRATE);
}

#endif

/* Example sketch ------------------------------------------------------------*/

// Instantiate ODrive objects
ODriveCAN odrv0(wrap_can_intf(can_intf), ODRV0_NODE_ID); // Standard CAN message
ID
ODriveCAN* odrives[] = {&odrv0}; // Make sure all ODriveCAN instances are
accounted for here

struct ODriveUserData {
    Heartbeat_msg_t last_heartbeat;
    bool received_heartbeat = false;
```

```
    Get_Encoder_Estimates_msg_t last_feedback;
    bool received_feedback = false;
};

// Keep some application-specific user data for every ODrive.
ODriveUserData odrv0_user_data;

// Called every time a Heartbeat message arrives from the ODrive
void onHeartbeat(Heartbeat_msg_t& msg, void* user_data) {
    ODriveUserData* odrv_user_data = static_cast<ODriveUserData*>(user_data);
    odrv_user_data->last_heartbeat = msg;
    odrv_user_data->received_heartbeat = true;
}

// Called every time a feedback message arrives from the ODrive
void onFeedback(Get_Encoder_Estimates_msg_t& msg, void* user_data) {
    ODriveUserData* odrv_user_data = static_cast<ODriveUserData*>(user_data);
    odrv_user_data->last_feedback = msg;
    odrv_user_data->received_feedback = true;
}

// Called for every message that arrives on the CAN bus
void onCanMessage(const CanMsg& msg) {
    for (auto odrive: odrives) {
        onReceive(msg, *odrive);
    }
}

void setup() {
    Serial.begin(115200);

    // Wait for up to 3 seconds for the serial port to be opened on the PC side.
    // If no PC connects, continue anyway.
    for (int i = 0; i < 30 && !Serial; ++i) {
        delay(100);
    }
    delay(200);

    Serial.println("Starting ODriveCAN demo");

    // Register callbacks for the heartbeat and encoder feedback messages
    odrv0.onFeedback(onFeedback, &odrv0_user_data);
    odrv0.onStatus(onHeartbeat, &odrv0_user_data);

    // Configure and initialize the CAN bus interface. This function depends on
    // your hardware and the CAN stack that you're using.
    if (!setupCan()) {
        Serial.println("CAN failed to initialize: reset required");
        while (true); // spin indefinitely
    }

    Serial.println("Waiting for ODrive...");
    while (!odrv0_user_data.received_heartbeat) {
        pumpEvents(can_intf);
        delay(100);
    }

    Serial.println("found ODrive");
```

```
    // request bus voltage and current (1sec timeout)
    Serial.println("attempting to read bus voltage and current");
    Get_Bus_Voltage_Current_msg_t vbus;
    if (!odrv0.request(vbus, 1)) {
        Serial.println("vbus request failed!");
        while (true); // spin indefinitely
    }

    Serial.print("DC voltage [V]: ");
    Serial.println(vbus.Bus_Voltage);
    Serial.print("DC current [A]: ");
    Serial.println(vbus.Bus_Current);

    Serial.println("Enabling closed loop control...");
    while (odrv0_user_data.last_heartbeat.Axis_State !=
ODriveAxisState::AXIS_STATE_CLOSED_LOOP_CONTROL) {
        odrv0.clearErrors();
        delay(1);
        odrv0.setState(ODriveAxisState::AXIS_STATE_CLOSED_LOOP_CONTROL);

        // Pump events for 150ms. This delay is needed for two reasons;
        // 1. If there is an error condition, such as missing DC power, the ODrive
might
        //     briefly attempt to enter CLOSED_LOOP_CONTROL state, so we can't rely
        //     on the first heartbeat response, so we want to receive at least two
        //     heartbeats (100ms default interval).
        // 2. If the bus is congested, the setState command won't get through
        //     immediately but can be delayed.
        for (int i = 0; i < 15; ++i) {
            delay(10);
            pumpEvents(can_intf);
        }
    }

    Serial.println("ODrive running!");
}

void loop() {
    pumpEvents(can_intf); // This is required on some platforms to handle incoming
feedback CAN messages

    float SINE_PERIOD = 2.0f; // Period of the position command sine wave in
seconds

    float t = 0.001 * millis();

    float phase = t * (TWO_PI / SINE_PERIOD);

    odrv0.setPosition(
        sin(phase), // position
        cos(phase) * (TWO_PI / SINE_PERIOD) // velocity feedforward (optional)
    );

    // print position and velocity for Serial Plotter
    if (odrv0_user_data.received_feedback) {
        Get_Encoder_Estimates_msg_t feedback = odrv0_user_data.last_feedback;
        odrv0_user_data.received_feedback = false;
        Serial.print("odrv0-pos:");
        Serial.print(feedback.Pos_Estimate);
```

```
        Serial.print(",");
        Serial.print("odrv0-vel:");
        Serial.println(feedback.Vel_Estimate);
    }
}
```

4.6.4 SDK de ROS

Los pasos siguientes se han probado y verificado en Ubuntu 23.04 con ROS 2 Iron. No se admiten las plataformas macOS ni Windows, y no se han verificado en otras versiones de ROS 2, en las que habría que hacer los ajustes oportunos.

4.6.4.1 Instalar el paquete odrive_can

1. Cree un nuevo espacio de trabajo de ROS 2 (véase `https://docs.ros.org/en/iron/index.html`).
2. Descargue el código con `git clone https://github.com/odriverobotics/odrive_can` en el directorio `src` de ese espacio de trabajo.
3. En un terminal, sitúese en la raíz del espacio de trabajo y ejecute:

   ```
   colcon build --packages-select odrive_can
   ```

4. Prepare el entorno antes de ejecutar:

   ```
   source ./install/setup.bash
   ```

5. Lance el nodo de ejemplo:

   ```
   ros2 launch odrive_can example_launch.yaml
   ```

4.6.4.2 Llamar a los servicios y ver los mensajes

Se supone que el nodo `odrive_can_node` anterior se ejecuta en el espacio de nombres `odrive_axis0` (configurable en `./launch/example_launch.yaml`). Una vez en marcha el nodo del apartado 4.6.4.1, pueden verse los mensajes de los tópicos publicados, por ejemplo:

```
ros2 topic echo /odrive_axis0/controller_status
ros2 topic echo /odrive_axis0/odrive_status
```

y llamar a las interfaces de servicio disponibles; por ejemplo, esta llamada inicia la calibración del motor:

```
ros2 service call /odrive_axis0/request_axis_state \
  /odrive_can/srv/AxisState "{axis_requested_state: 4}"
```